

Analysis of algorithm

Proof of correctness

- The algorithm is complete/correct: the post-condition is respected on all possible inputs satisfying the pre-condition
 - Precondition: a predicate I on the input data
 - Postcondition: a predicate O on the output data
 - Correctness: proving $I \Rightarrow O$
-
- The algorithm terminates: For all possible input, the algorithm exits

Loop invariants

- *A loop invariant is a logic formula or just a set of rules, that is true before, during and after the loop in question (so it's unbiased to iteration). It is imperative for it to contain rules for all the variables that occur in the said loop because we need to **bind** them all to the set of values we want them to be.*
- A loop invariant can be as complicated and as simple as you want it to be. However, the point is that it should be constructed to resemble the problem at hand as closely as possible.

Loop invariants

- **Initialization:** It is true prior to the first iteration of the loop.
- **Maintenance:** If it is true before an iteration of the loop, it remains true before the next iteration
- **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

Example

$x = 10$

$y = 4$

$z = 0$

$n = 0$

while($n < x$):

$z = z + y$

$n = n + 1$

Example

- Logically, this code just calculates the value $x*y$ and stores it in z , this means that $z = x*y$.
- $z = n*y$

Before loop

$x = 5$

$y = 4$

$z = 0$

$n = 0$

- RULE 1: $z == n * y$
 $0 == 0 * 4 = 0 \iff \text{True}$
so RULE 1 applies

Maintenance

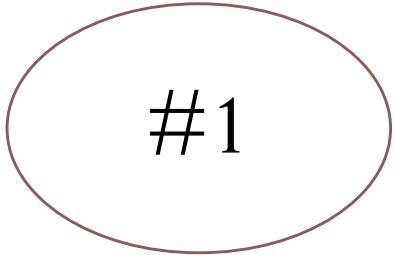
$x = 5, y = 4$ while $(0 < 5)$ $z = 0 + 4; n = 0 + 1;$

So $z' = 4$

So $n' = 1$

After

- RULE 1: $z' == n * y$
 $4 == 1 * 4 = 4 \iff \text{True}$
so RULE 1 applies



#1

Maintenance

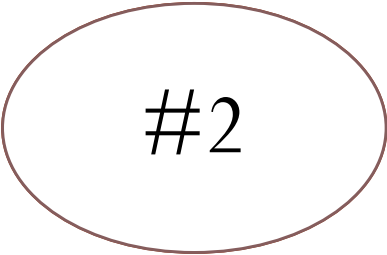
$x = 5, y = 4$

$z' = 8$

$N' = 2$

After

- RULE 1: $z == n * y$
 $8 == 2 * 4 = 8 \iff \text{True}$
so RULE 1 applies



#2

Maintenance

Iterations Contd.

Termination

$N' = 5$ while ($5 < 5$) loop fails, so n and z wont change

- RULE 1: $z == n * y$

$$20 == 5 * 4 = 20 \iff \text{True}$$

so RULE 1 applies

Insertion sort

INSERTION-SORT(A)

1 *for* $j = 2$ *to* $A.length$

2 $key = A[j]$

3 *// Insert* $A[j]$ *into the sorted sequence* $A[1 .. j - 1]$.

4 $i = j - 1$

5 *while* $i > 0$ *and* $A[i] > key$

6 $A[i+1] = A[i]$

7 $i = i - 1$

8 $A[i+1] = key$

Loop Invariant

- $A[1..j-1]$ constitutes the currently sorted set.
- $A[j+1..N]$ corresponds to the remaining elements in array.
- Loop invariant : At the start of each iteration of the **for** loop of lines 1–8, the subarray $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$, but in sorted order.

Initialization

- **Initialization:** We start by showing that the loop invariant holds before the first loop iteration, when $j = 2$. The subarray $A[1 \dots j - 1]$, therefore, consists of just the single element $A[1]$, which is in fact the original element in $A[1]$.
- Moreover, this subarray is sorted, which shows that the loop invariant holds prior to the first iteration of the loop.

Maintenance

- **Maintenance:** Each iteration maintains the loop invariant.
- The body of the **for** loop works by moving $A[j - 1]$, $A[j - 2]$, $A[j - 3]$, and so on by one position to the right until it finds the proper position for $A[j]$ (lines 4–7), at which point it inserts the value of $A[j]$ (line 8).
- The subarray $A[1 \dots J]$ then consists of the elements originally in $A[1 \dots J]$, but in sorted order.
- Incrementing j for the next iteration of the **for** loop then preserves the loop invariant.

Termination

- **Termination:** Finally, we examine what happens when the loop terminates.
- The condition causing the **for** loop to terminate is that $j > A.length = n$. Because each loop iteration increases j by 1.
- The subarray $A[1 \dots n]$ consists of the elements originally in $A[1 \dots N]$, but in sorted order.
- Observing that the subarray $A[1 \dots n]$ is the entire array, we conclude that the entire array is sorted.
- Hence, the algorithm is correct.